DRIFTLESS — Frontend Architecture & UI Interaction Workflows

1. Overview of Frontend Architecture

The DRIFTLESS frontend consists of two primary web templates served by Django:
1. `dashboard/core/templates/auth.html`: Unified authentication portal handling registration, login, persona selection, and token display.
2. `dashboard/core/templates/dashboard.html`: Single-page analytics dashboard providing KPI scorecards, interactive Chart.js visualizations, per-site distraction management, Focus Mode controls, category overrides, and data privacy options.

Both interfaces use a modern hybrid frontend stack:
- **Styling:** Vanilla CSS Custom Properties (`theme.css`) defining the design system, combined with Tailwind CSS (loaded via CDN) for responsive layouts and micro-components.
- **Client Logic:** Vanilla ES6+ JavaScript embedded directly within `<script>` blocks in the templates, avoiding complex build steps or node packaging overhead.
- **Data Layer:** Dynamic asynchronous REST communication with Django using the native browser `fetch` API and token authentication headers.

---

2. Page Structure & Navigation

2.1. Dashboard Layout Structure

The dashboard uses an application shell layout:
- **Left Desktop Sidebar (`<aside>`):**
- Brand header with Driftless logo and application badge.
- Primary navigation links:
- *Dashboard* (`#tab-dashboard`): Overview KPIs, Focus Goal Ring, 7-day chart, distractions list, detailed logs.
- *Analytics Detail* (`#tab-analytics`): Extended tabular logs and category breakdown.
- *Settings & Rules* (`#tab-settings`): Category override manager, API token copy/regenerate tool, and data export/deletion.
- Bottom user profile pill with active persona tag and sign-out button.
- **Top Header Bar (`<header>`):**

- Mobile hamburger toggle button.
- Active section title (`Dashboard` / `Analytics Detail` / `Settings & Rules`).
- Dynamic greeting based on current local time (`Good morning / afternoon / evening, <username>` 👋).
- Current calendar date (`headerDate`).
- Data Mode Switcher (`Demo` vs. `Live` toggle pill).
- Theme Toggle Button (`dashThemeToggle`) with animated ☀/ 🌙 SVG icons.
- **Main Scrollable Content (`<main>`):**
- Section tabs toggled via CSS `.tab-content.active` classes.
- Modal overlay dialogs for Per-Site Limits (`#limitsModal`) and mobile sidebar drawer.

---

3. Styling & Dual-Theme System

DRIFTLESS implements two distinct, synchronized color themes across the frontend:

1. **Calm Sky (Light Mode):** Clean, high-legibility palette tailored for daylight focus.
- Background (`--bg`): `#F3F7FD` (Soft glacial blue-grey)
- Cards (`--card`): `#FFFFFF`
- Primary Brand (`--pri`): `#6463E0` (Deep indigo)
- Accent (`--accent`): `#60B0E7` (Sky blue)
- Text Ink (`--ink`): `#1B2340` (Midnight navy)
- Category Indicators: Productivity (`#10B981`), Social (`#F59E0B`), Entertainment (`#F43F5E`), Other (`#94A3B8`)

2. **Opal Noir (Dark Mode):** Sleek, OLED-friendly low-light palette tailored for late-night productivity.
- Background (`--bg`): `#0A0A0A` (Deep pitch black)
- Cards (`--card`): `#141416` (Smoky obsidian)
- Primary Accent (`--pri`): `#E2C9FF` (Luminous lilac)
- Secondary Accent (`--accent`): `#8CFFDD` (Mint luminescence)
- Text Ink (`--ink`): `#FFFFFF` (Pure white)
- Muted Ink (`--ink-muted`): `#8E8E93`

Flash-of-Unthemed-Content (FOUC) Prevention

To prevent bright flashes during page loads, both `auth.html` and `dashboard.html` execute an inline, synchronous JavaScript snippet immediately inside `<head>` prior to CSS or body rendering:

```
<script>
  (function() {
    var t = localStorage.getItem('driftless-theme') || 'light';
    document.documentElement.setAttribute('data-theme', t);
  })();
</script>
```

When toggled, `dashThemeToggle` calls `document.documentElement.setAttribute('data-theme', newTheme)` and saves the preference to `localStorage.setItem('driftless-theme', newTheme)`. All CSS variables cascade instantaneously without requiring a page reload.

---

4. Charts & Analytics Visualization

The 7-Day Trend Chart uses Chart.js 4.4.3 mounted on `<canvas id="weeklyChart">`.

- **Dynamic CSS Variable Integration:**

Rather than hardcoding hex colors in JavaScript, `renderChart()` reads theme colors dynamically at runtime:

```
javascript const style = getComputedStyle(document.documentElement); const cProd = style.getPropertyValue('--prod').trim() || '#10B981'; const cSoc = style.getPropertyValue('--soc').trim() || '#F59E0B'; const cEnt = style.getPropertyValue('--ent').trim() || '#F43F5E'; const cOther = style.getPropertyValue('--other').trim() || '#94A3B8';
```

When the theme toggle is clicked, `renderChart()` re-executes, instantly redrawing grid lines, tooltips, and bar colors to match Calm Sky or Opal Noir.

- **Chart Type Toggle:** Users can switch seamlessly between stacked bar format (`currentChartType = 'bar'`) and multi-line area format (`currentChartType = 'line'`).
- **Focus Goal Ring:** An SVG circular progress bar calculates today's productive hours against the user's daily target (`focus_goal_minutes` , default 180m = 3h) via `stroke-dasharray` .

---

5. End-to-End User Interaction Workflows

Workflow 1: Data Mode Switching (Demo vs. Live)

```
User Action: Clicks "Live" on top-nav toggle (#dataModeSwitcher)
  ↓
Event Handler: onclick="setDataMode('live')"
  ↓
Function Execution:
  1. Updates active button styling.
  2. Calls loadData().
  ↓
Data Source:
  GET /api/summary/ with Authorization: Token <api_token>
  ↓
UI Update:
  - If successful: renderDashboard(data, false) displays live SQLite database records.
  - If network fails: console.warn logged and loadDemoFallback() displays 14-day reference data.
```

Workflow 2: Adding a Per-Site Daily Limit

```
User Action: Enters "reddit.com", "25", clicks "Save Limit" in modal
  ↓
Event Handler: onclick="submitNewLimit()"
  ↓
Function Execution:
  1. Reads values from modalAddDomain and modalAddMins.
  2. Executes fetch('/api/limits/', { method: 'POST', body: JSON.stringify({ ... }) }).
  ↓
Data Source:
  Django SiteLimit model creates/updates record for request.user.
  ↓
UI Update:
  loadModalLimits() refreshes modal list; closeLimitsModal() re-fetches /api/summary/ and
updates distraction progress bars.
```

Workflow 3: Toggling Focus Mode from Dashboard

```
User Action: Flips toggle switch (#focusToggle)
  ↓
Event Handler: onchange="toggleFocusMode(this.checked)"
  ↓
Function Execution:
  1. applyFocusUI(enabled) immediately turns card green and activates pulsing dot.
  2. Executes fetch('/api/settings/', { method: 'PATCH', body: JSON.stringify({
focus_mode_active: enabled }) }).
  ↓
Data Source:
  Django UserSettings.focus_mode_active is updated in database.
  ↓
Downstream Effect:
  Next periodic pull by Chrome extension activates DNR dynamic rules to block distracting
sites.
```

Workflow 4: Custom Category Override

```
User Action: Enters "reddit.com", selects "Productivity", clicks "Add Override"
  ↓
Event Handler: onclick="addCategoryOverride()"
  ↓
Function Execution:
  Executes fetch('/api/categories/', { method: 'POST', body: JSON.stringify({ domain:
'reddit.com', category: 'Productivity' }) }).
  ↓
Data Source:
  Django CategoryOverride record created; retroactively updates past UsageLog entries for
that domain.
  ↓
UI Update:
  loadCategoryOverrides() displays new override; loadData() recalculates Focus Efficiency
and redrew weekly chart.
```

Workflow 5: 1-Click Data Sovereignty

- **Data Export:** User clicks "Export All Data (JSON)" → Calls `window.open('/api/data/export/', '_blank')` → Browser downloads formatted JSON file containing all usage logs, site limits, and focus sessions.
- **Data Erase:** User clicks "Erase Browsing Data" → Displays confirmation dialog → Calls `POST /api/data/delete/` → Server purges all `UsageLog` and `FocusSession` entries → Dashboard reloads with zeroed metrics.

---

6. Data Provenance (Origin of Displayed Values)

UI Element	In Demo Mode	In Live Mode	Source
KPI: Focus Efficiency	Static calculated (74%)	Dynamic (`round(prod / (prod+soc+ent) * 100)`)	Backend `/api/summary/`
KPI: Productive Time	Static (200m / 3.3h)	Sum of today's `Productivity` logs	Backend `/api/summary/`
KPI: Screen Time	Static (270m / 4.5h)	Sum of all today's domain logs	Backend `/api/summary/`
KPI: Streak	Static (14 Days)	Calculated strict consecutive days	Backend `/api/summary/`
Weekly Chart Data	Static 7-day array	Aggregated past 7 calendar days	Backend `/api/summary/`
Distraction Progress Bars	3 preset rows	Merged `SiteLimit` + today's usage logs	Backend `/api/summary/`

UI Element	In Demo Mode	In Live Mode	Source
Detailed Log Table	5 preset sites	All domain logs recorded today	Backend `/api/summary/`
Active Theme	Client `localStorage`	Client `localStorage`	Browser `localStorage`
API Token	Django Template Context	Django Template Context	Injected into HTML via `{{ api_token }}`